

Create your Amazon ElastiCache cluster

Cluster engine: **Redis**
 In-memory data structure store used as database, cache and message broker. ElastiCache for Redis offers built-in Auto Failover and enhanced security.
☐ Cluster Mode enabled

☐ Memcached
 High performance, distributed memory object caching system, intended for use in speeding up dynamic web applications.

Redis settings

Name:

Description:

Engine version compatibility: 3.2.10

Port: 6379

Parameter group: default.redis3.2

Node type: cache.m1.large (10.5 GB)

Number of replicas: 2

Click launch to start the database cluster.

[Note: Just like DynamoDB there are a number of advanced settings that you can tweak here, such as shard count, replication factor etc, but for the purposes of this tutorial let's keep this at default settings] Once launched the Redis cluster is available on its configured port (default is 6379). To connect to it via aws-cli from a machine, enter the following command:

- `>redis-cli -h <hostname> -p <port>`
- Eg: `redis-cli -h test-cache.us-east-1.cache.amazonaws.com -p 6379`

This should allow you to login to the remote Redis server using the REPL (Read,Eval,Print and Loop) mode, where you can type further commands to modify data.

You have now successfully setup a managed Redis cluster, that can be used in all applications by specifying the given host and port. The cluster is fully managed, including auto scaling and distribution of shards and can be made available over multiple AZs.

In this chapter we saw how to install a database server on a barebone Linux machine and connect to it. We also worked on setting up our own managed relational database on Amazon RDS, a non-relational database on DynamoDB as well an in-memory database on AWS ElastiCache. Hopefully, we now have a better idea on how we can leverage the different database technology services offered by Amazon for our use cases. Needless to say, it is worth spending the time and effort to look at the pricing of each individual service on offer and compare it with alternatives (eg: own DB on EC2 vs managed RDS instance) for the optimum choice in each case. **d**